COMP 4003

Design and Analysis of Algorithms

Lecture 1:

Logistics, introduction, and multiplication!

Today

- Why are you here? 
- Course overview, logistics, and how to succeed in this course.
- Some actual computer science.

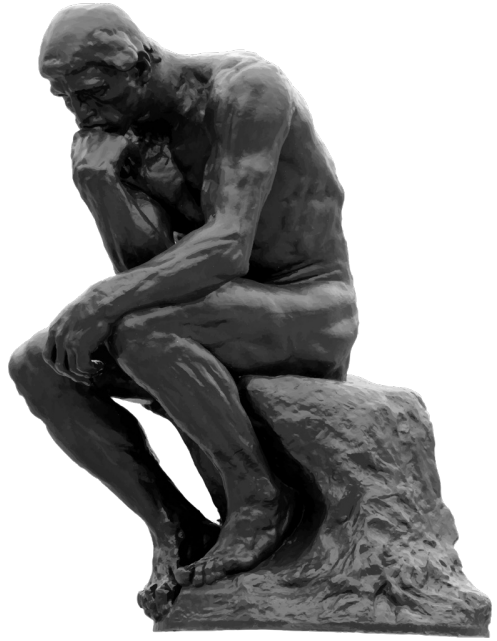
Why are you here?

You are better equipped to answer this question than I am, but I'll give it a go anyway…

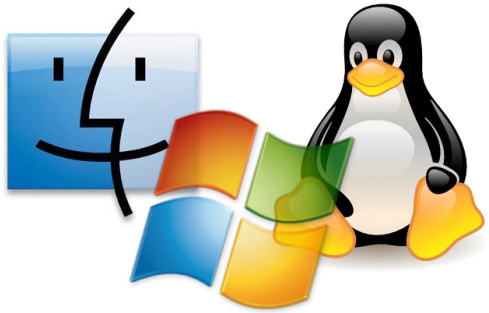
- Algorithms are fundamental.
- Algorithms are useful.
- Algorithms are fun!
- COMP 4003 is a required course.

Why is COMP 4003 required?

- Algorithms are fundamental.
- Algorithms are useful.
- Algorithms are fun!



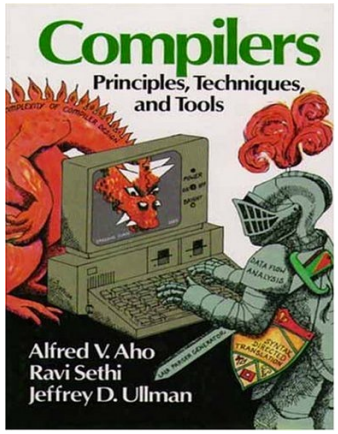
Algorithms are fundamental



Operating Systems

Machine learning

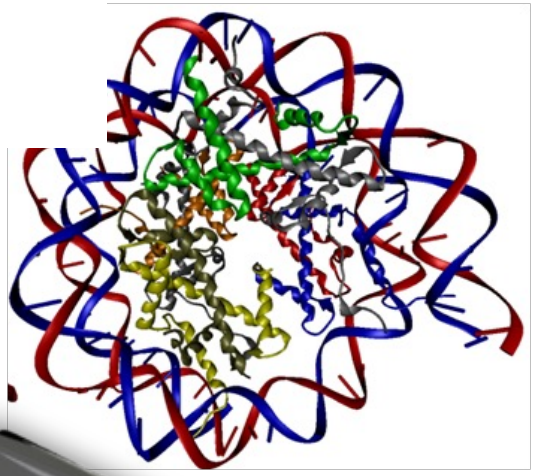
Cryptography



Compilers

The Computational Lens

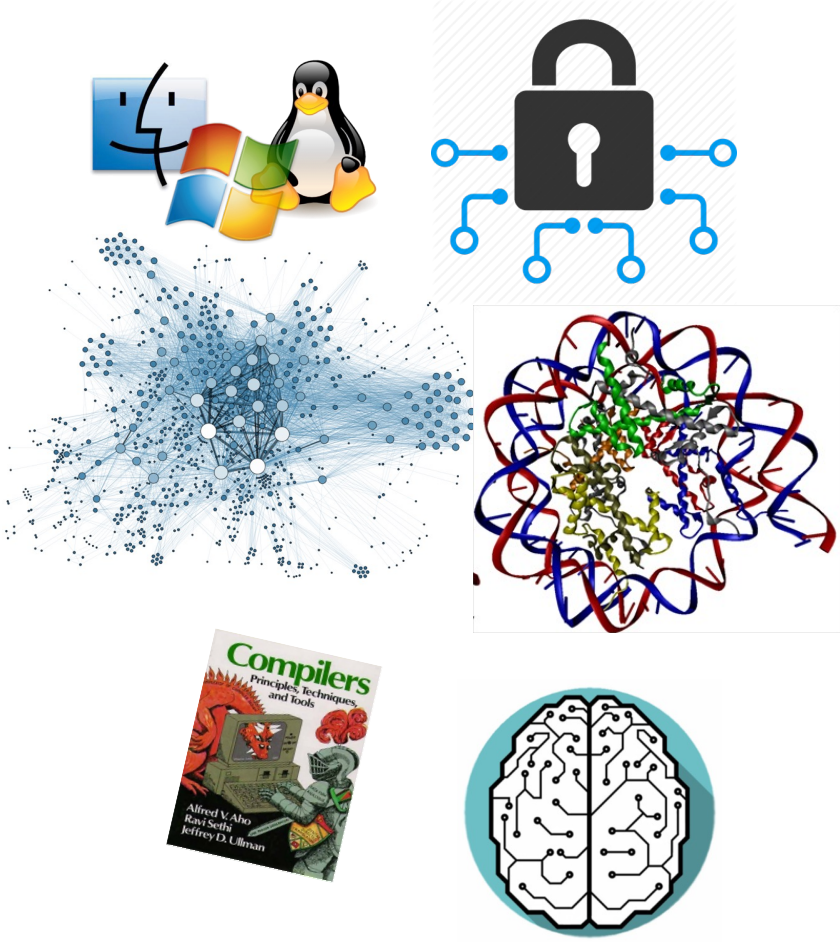
Networking



nal Biology

Algorithms are useful

- All those things, without CENG class numbers
- As we get more and more data and problem sizes get bigger and bigger, algorithms become more and more important.
- Will help you get a job.



Algorithms are fun!

- Algorithm design is both an art and a science.
- Many surprises!
- A young field, lots of exciting research questions!

Today

- Why are you here?
- Course overview, logistics, and how to succeed in this course.
- Some actual computer science.

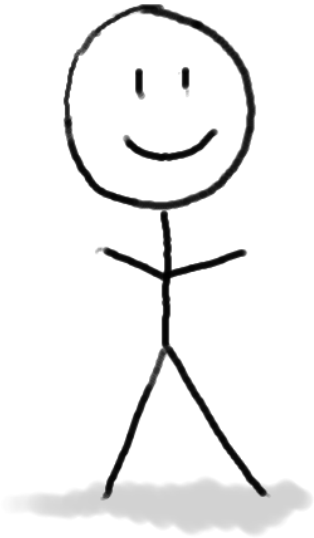
Course goals

- The design and analysis of algorithms
  - These go hand-in-hand

- In this course you will:
  - Learn to think analytically about algorithms
  - Flesh out an "algorithmic toolkit"
  - Learn to communicate clearly about algorithms

The algorithm designer's question

Can I do better?



Algorithm designer

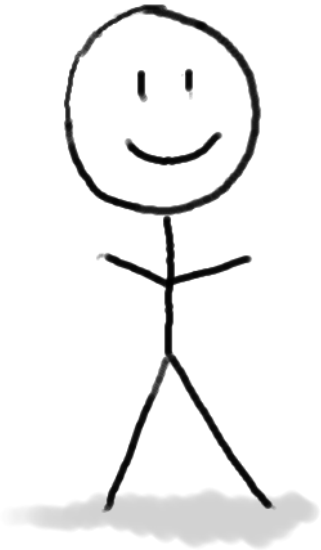
The algorithm designer's internal monologue…

What exactly do we mean by better? And what about that corner case? Shouldn't we be zero-indexing?

Plucky the
Pedantic Penguin

Detail-oriented
Precise
Rigorous

Can I do better?



Algorithm designer

Dude, this is just like that other time. If you do the thing and the stuff like you did then, it'll totally work real fast!

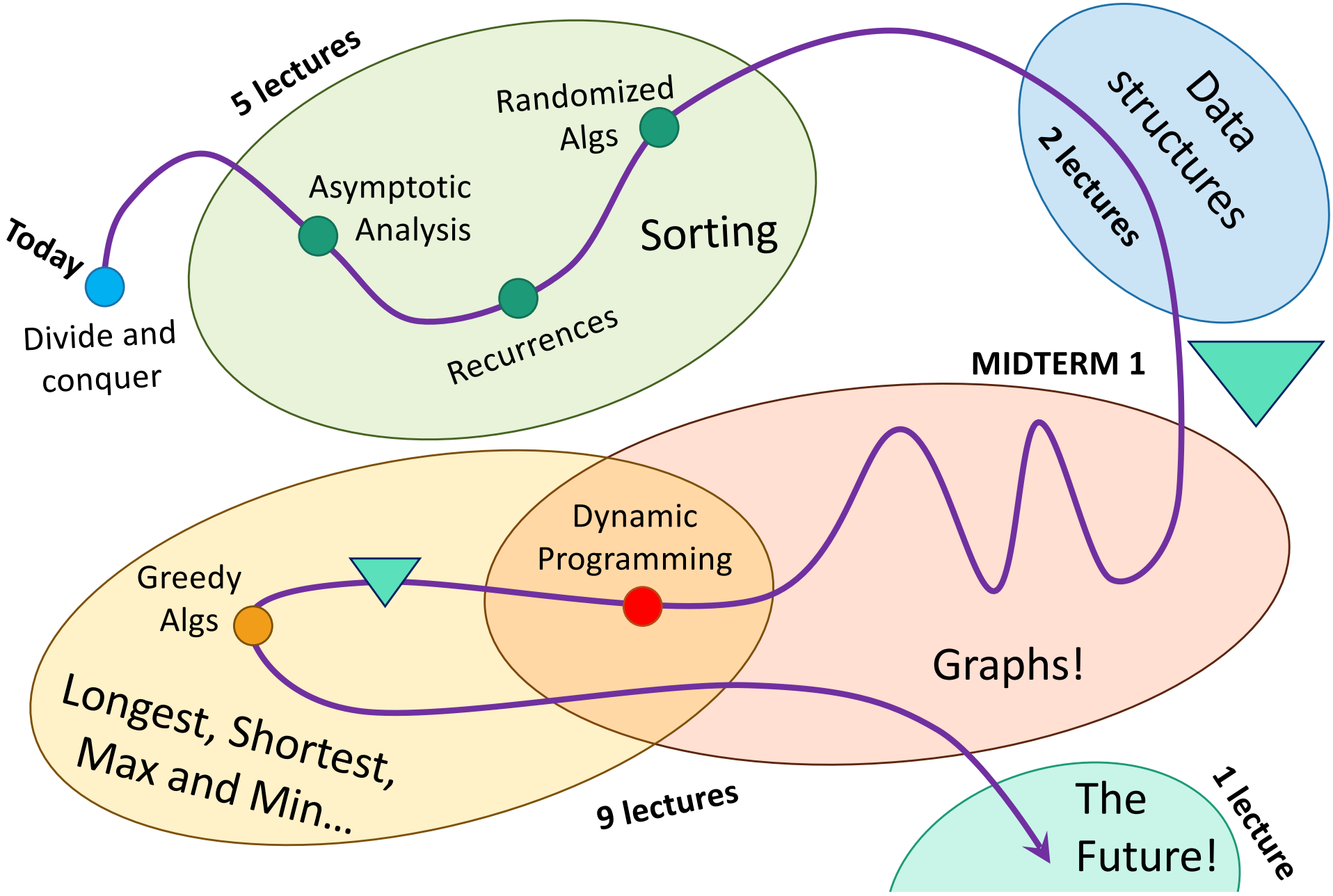
Lucky the
Lackadaisical Lemur

Big-picture
Intuitive
Hand-wavey

Both sides are necessary!

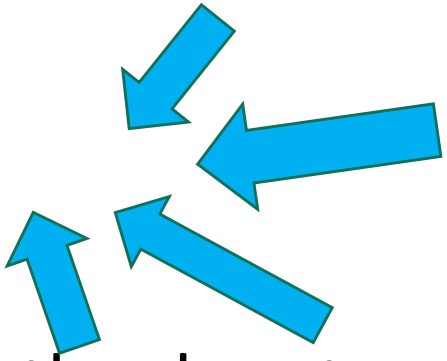
More detailed schedule on the website!

Roadmap



Course elements and resources

- Course website:



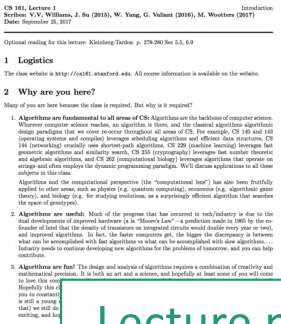
- Lectures
- Office hours, textbooks, etc
- Midterm: 30%
- Project: 30%
- Final: 30%
- Quiz(es): 10%

Lectures

- Right here, Tue, 19:00-20:30!
- Resources available:
  - Slides, Lecture Notes

Midterm: 30%

Slides are the slides from lecture.



CS 161, Lecture 1 — Introduction
Scribes: V.V. Williams, J. Su (2015), W. Yang, G. Valiant (2016), M. Wootters (2017)
Date: September 25, 2017

Optional reading for this lecture: Kleinberg-Tardos: p. 278-280 Sec 5.3, 6.9

1 Logistics

The class website is http://cs161.stanford.edu. All course information is available on the website.

2 Why are you here?

Many of you are here because the class is required. But why is it required?

1. **Algorithms are fundamental to all areas of CS:** Algorithms are the backbone of computer science. Wherever computer science reaches, an algorithm is there, and the classical algorithms algorithmic design paradigms that we cover re-occur throughout all areas of CS. For example, CS 140 and 143 (operating systems and compilers) leverages scheduling algorithms and efficient data structures, CS 144 (networking) crucially uses shortest-path algorithms, CS 229 (machine learning) leverages fast geometric algorithms and similarity search, CS 255 (cryptography) leverages fast number theoretic and algebraic algorithms, and CS 262 (computational biology) leverages algorithms that operate on strings–and often employs the dynamic programming paradigm. We'll discuss applications to all these subjects in this class.

   Algorithms and the computational perspective (the "computational lens") has also been fruitfully applied to other areas, such as physics (e.g. quantum computing), economics (e.g. algorithmic game theory), and biology (e.g. for studying evolutions, as a surprisingly efficient algorithm that searches the space of genotypes).

2. **Algorithms are useful:** Much of the progress that has occurred in tech/industry is due to the dual developments of improved hardware (a la "Moore's Law" – a prediction made in 1965 by the co-founder of Intel that the density of transistors on integrated circuits would double every year or two), and improved algorithms. In fact, the faster computers get, the bigger the discrepancy is between what can be accomplished with fast algorithms vs what can be accomplished with slow algorithms. . . Industry needs to continue developing new algorithms for the problems of tomorrow, and you can help contribute.

3. **Algorithms are fun!** The design and analysis of algorithms requires a combination of creativity and mathematical precision. It is both an art and a science, and hopefully at least some of you will come to love this cour
   Hopefully this c
   you to constantl
   is still a young
   that) we still do
   exciting, and lo

Lecture notes have math details that slides may omit

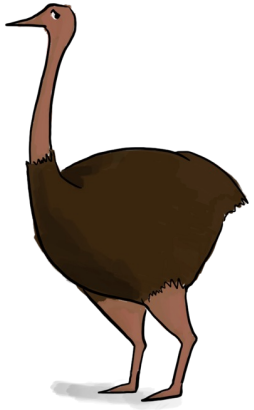
- Goal of lectures:
  - Hit the most important points of the week's material
    - Sometimes high-level overview
    - Sometimes detailed examples

How to get the most out of lectures

- **During lecture:**
  - Show up, ask questions, put your phone away.
  - May be helpful: take notes on printouts of the slides.
- **After lecture:**
  - Go through the exercises on the slides.

These guys will pop up on the slides and ask questions – those questions are for you!

Siggi the Studious Stork
(recommended exercises)



Ollie the Over-achieving Ostrich
(challenge questions)

- **Between lectures – more resources:**
  - Go to sections
  - Read textbook/notes
  - Do the homework

Exams

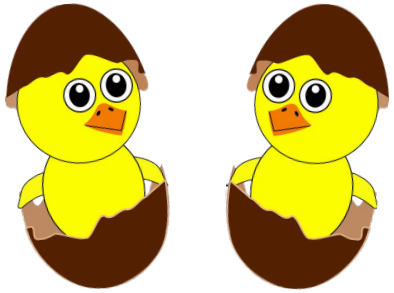
- There will be **one midterm** and a **final**.
  - **MIDTERM 1:** ?, take home
  - **FINAL:** take home

Everyone can succeed in this class!

smart

1. Work ~~hard~~
2. Ask for help
3. Help others

   (Think)-Pair-Share:



   2 min:
   Talk to person(s) next to you;
   tell them they'll succeed in this class,
   and that you're here help them!

4. OK… also work hard

Everyone can succeed in this class!

Most importantly:

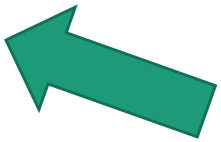
Don’t just find the solution.

Make sure you really *understand*:

- *Why is this the solution?*
- *What are the mental steps to coming up with the solution?*

Today

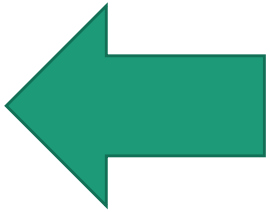
- Why are you here?
- Course overview, logistics, and how to succeed in this course.
- Some actual computer science.



Course goals

- Think analytically about algorithms
- Flesh out an "algorithmic toolkit"
- Learn to communicate clearly about algorithms

Today's goals

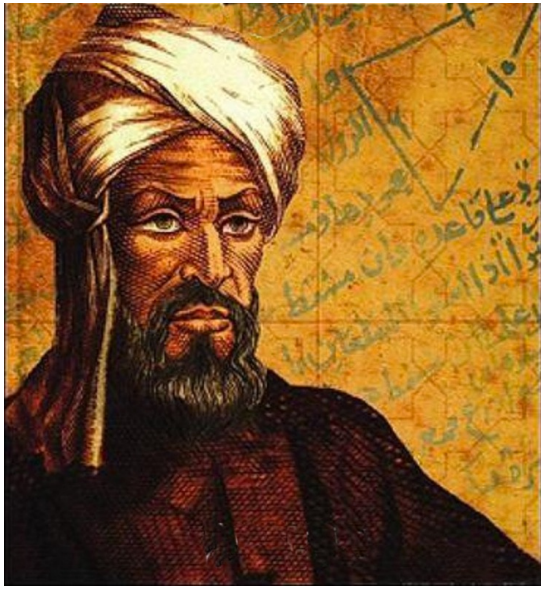


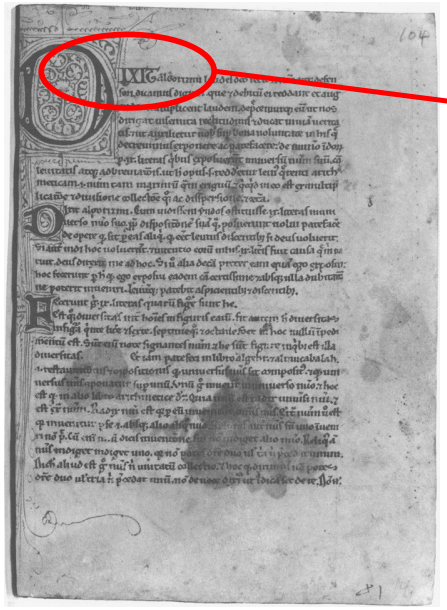
- Karatsuba Integer Multiplication
- Technique: Divide and conquer
- Meta points:
  - How do we measure the speed of an algorithm?

Let’s start at the beginning

Etymology of “Algorithm”

- Al-Khwarizmi was a 9th-century scholar, born in present-day Uzbekistan, who studied and worked in Baghdad during the Abbassid Caliphate.
- Among many other contributions in mathematics, astronomy, and geography, he wrote a book about how to multiply with Arabic numerals.
- His ideas came to Europe in the 12th century.





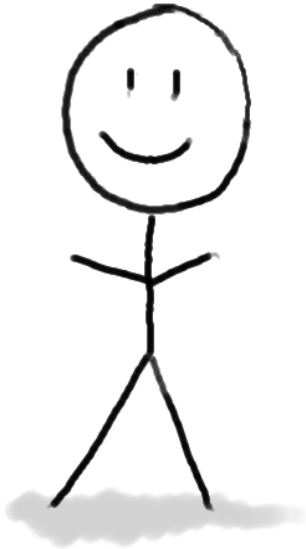
Dixit algorizmi
(so says Al-Khwarizmi)

- Originally, “Algorisme” [old French] referred to just the Arabic number system, but eventually it came to mean “Algorithm” as we know today.

This was kind of a big deal

XLIV × XCVII = ?

$$\begin{array}{r} 44 \\ \times\ 97 \\ \hline \end{array}$$



Integer Multiplication

$$\begin{array}{r} 44 \\ \times\ 97 \\ \hline \end{array}$$

Integer Multiplication

$$
\begin{array}{rr}
 & 12345678959314\,13 \\
\times & 4563823520395533 \\
\hline
\end{array}
$$

Integer Multiplication

n

```
  12339257207527523846237642835683649183745238562 98
x 45623235823423952856234672350191307501353500137 53
```

???

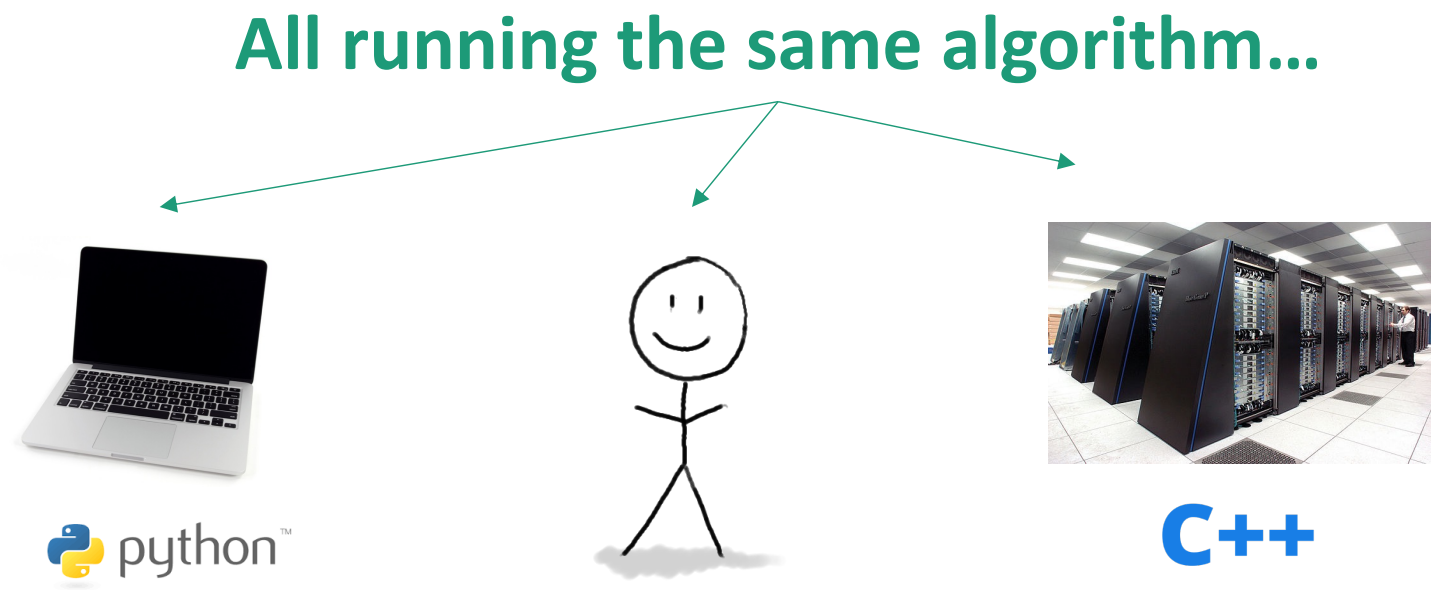
How long would this take you?

About n^2 one-digit operations

At most n^2 multiplications,
and then at most n^2 additions (for carries)
and then I have to add n different 2n-digit numbers…

Is that a useful answer?

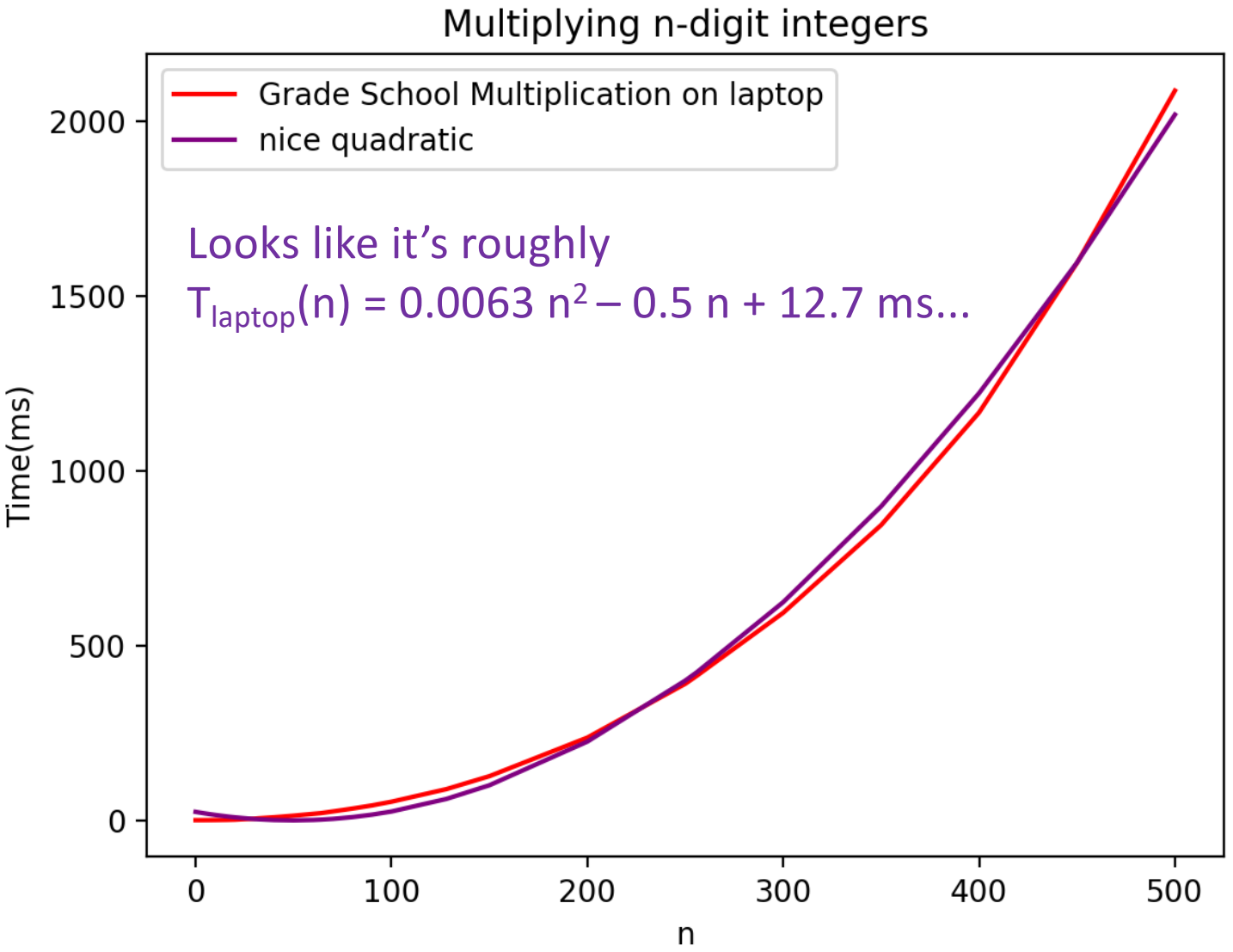
- How do we measure the runtime of an algorithm?



- We measure how the runtime scales with the size of the input.

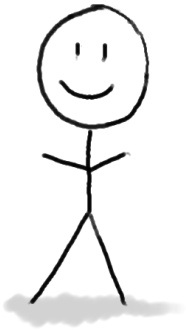
For grade school multiplication,

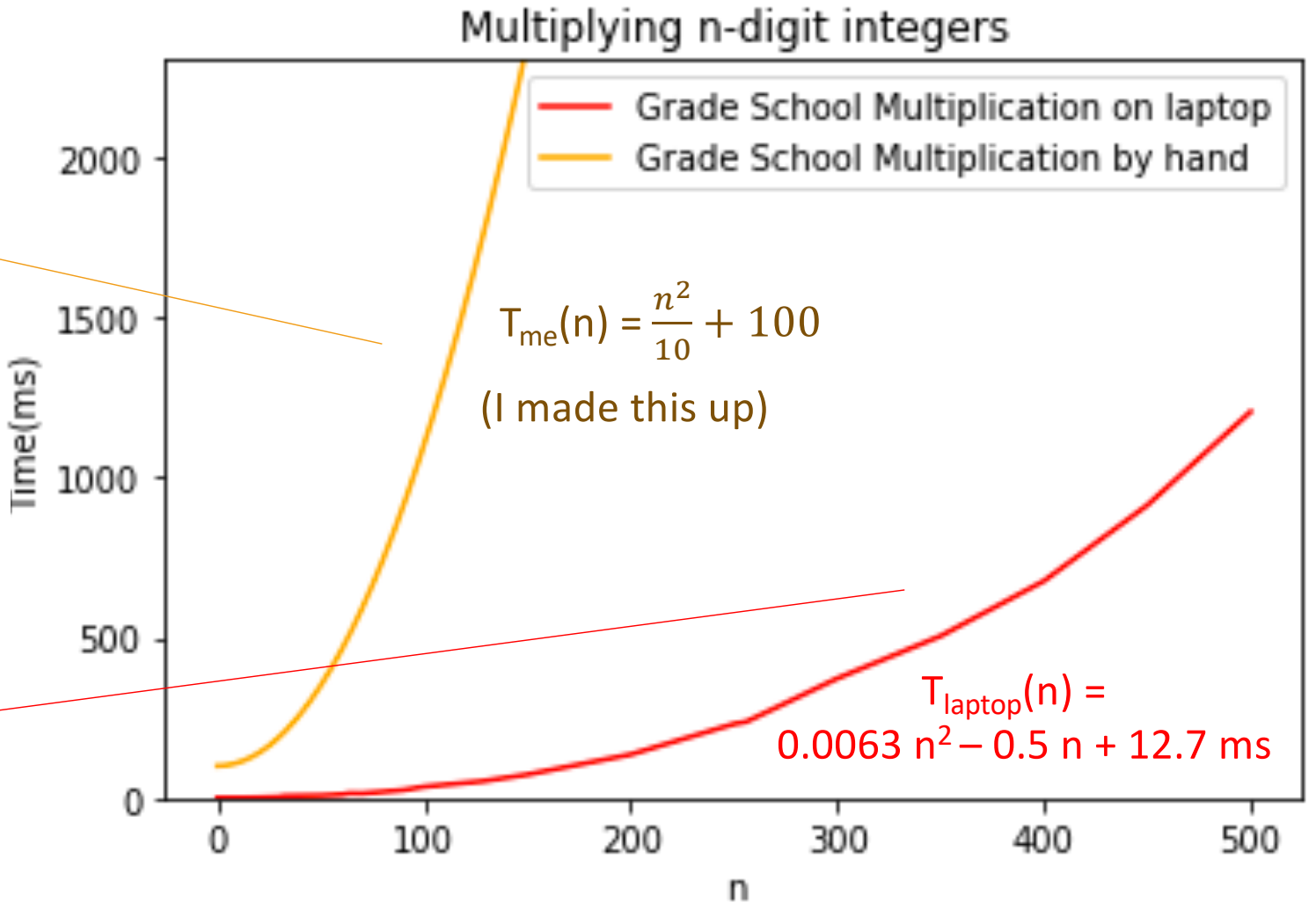
with python, on your laptop…



I am a bit slower than my laptop

But the runtime scales like n^2 either way.





python™

Asymptotic analysis

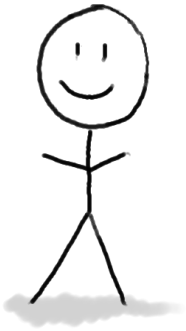
- How does the runtime scale with the size of the input?
  - Runtime of grade school multiplication scales like $\mathbf{n^2}$

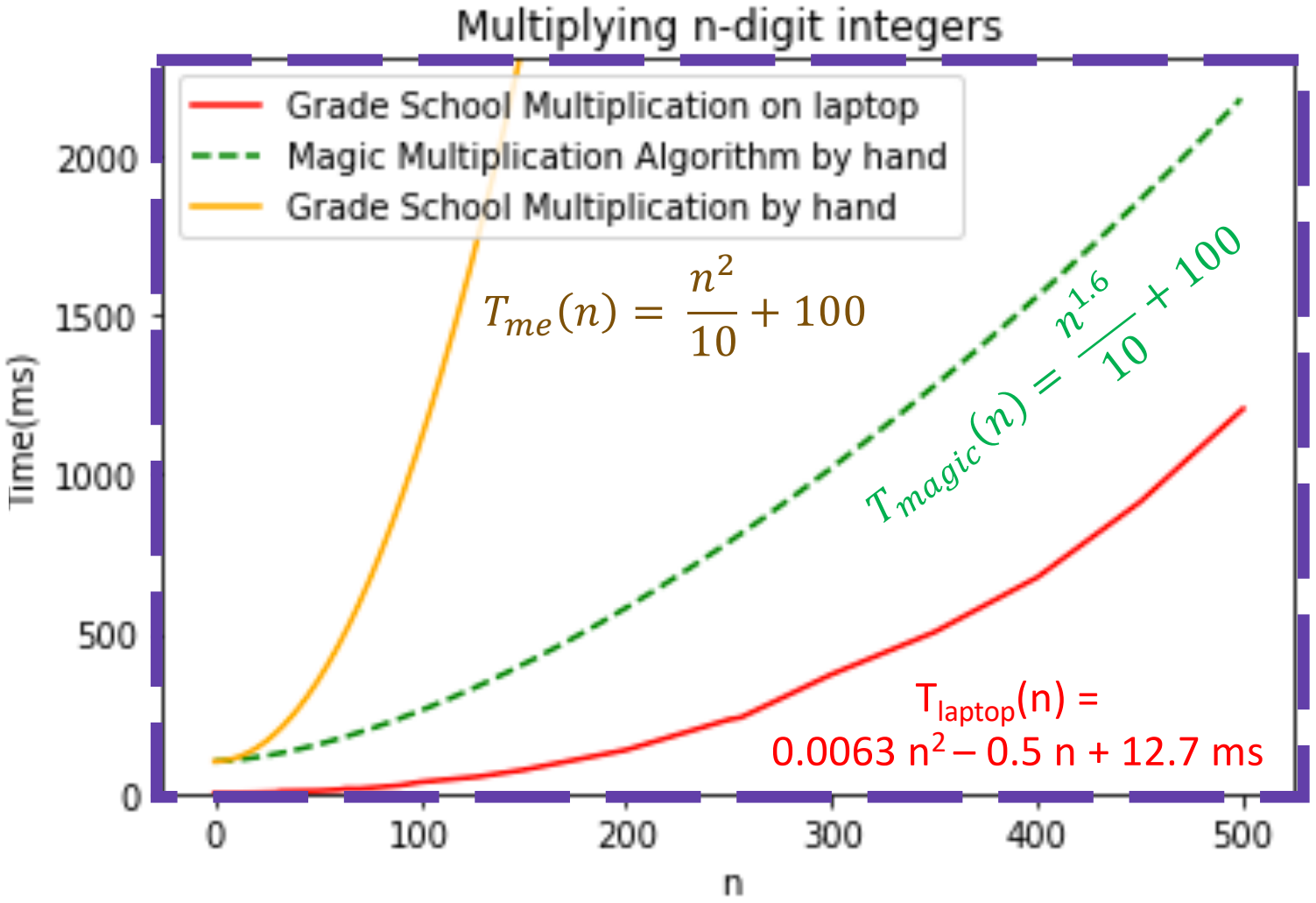
- We'll see a more formal definition on Thursday

Is this a useful answer?

Hypothetically…

A magic algorithm that scales like $n^{1.6}$

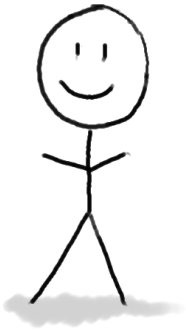


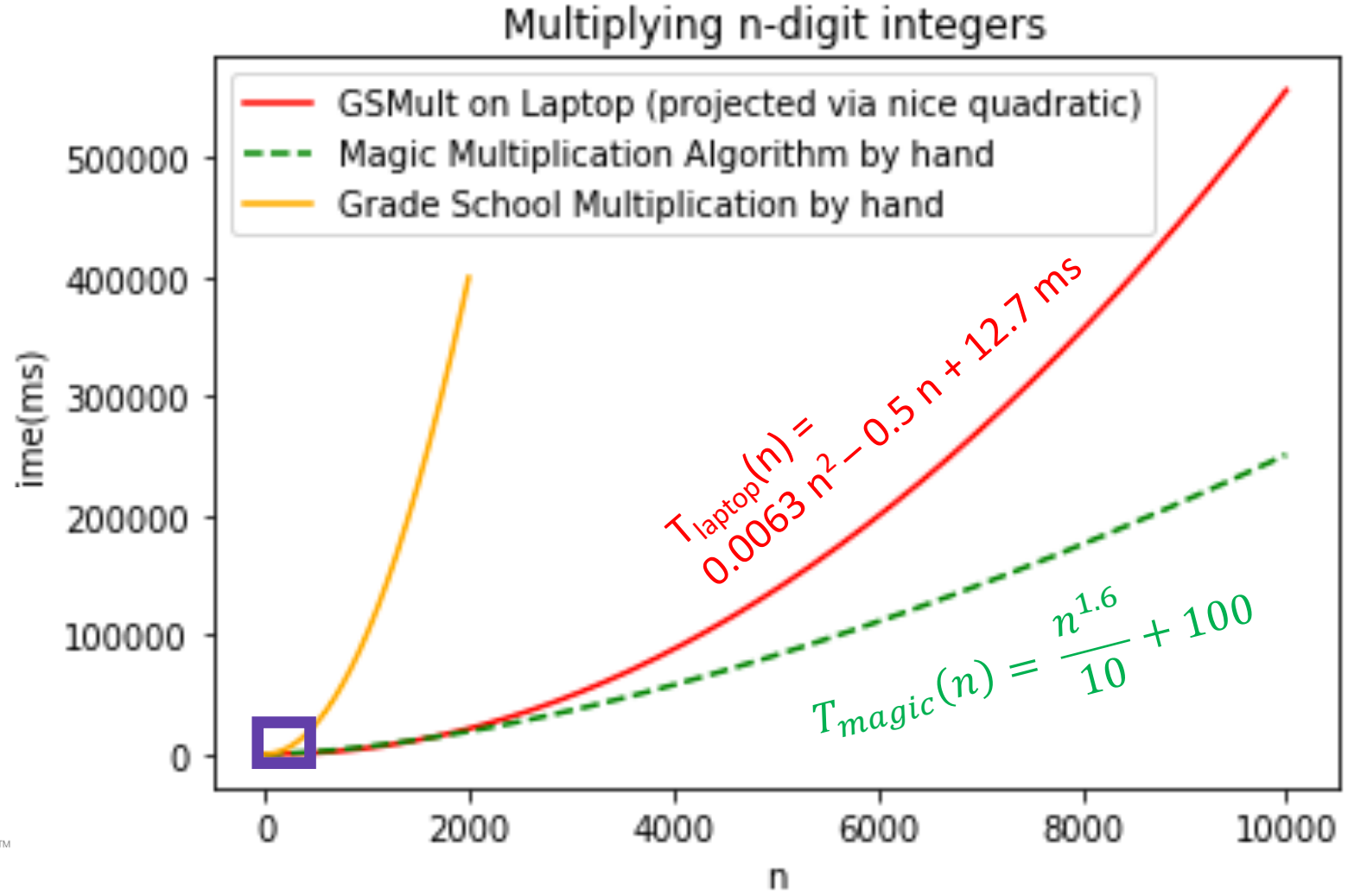


python™

Let n get bigger…

For large enough n, **it would be faster to do the magic algorithm by hand than the grade school algorithm on a computer!**





python™

Asymptotic analysis
is a useful notion…

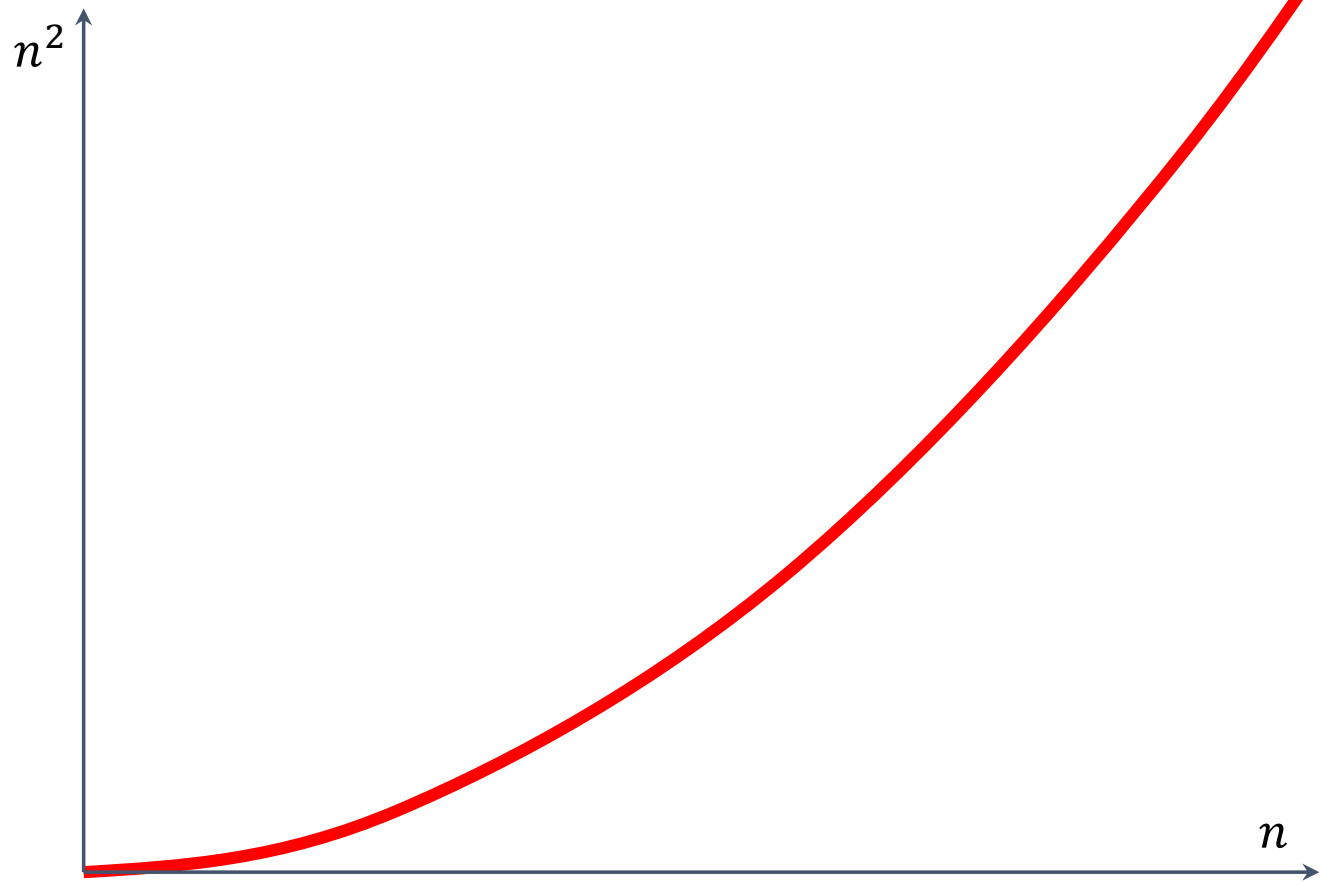
- How does the runtime scale with the size of the input?

- This is our measure of how “fast” an algorithm is.
- We’ll see a more formal definition Next Time

- So the question is…

Can we do better?

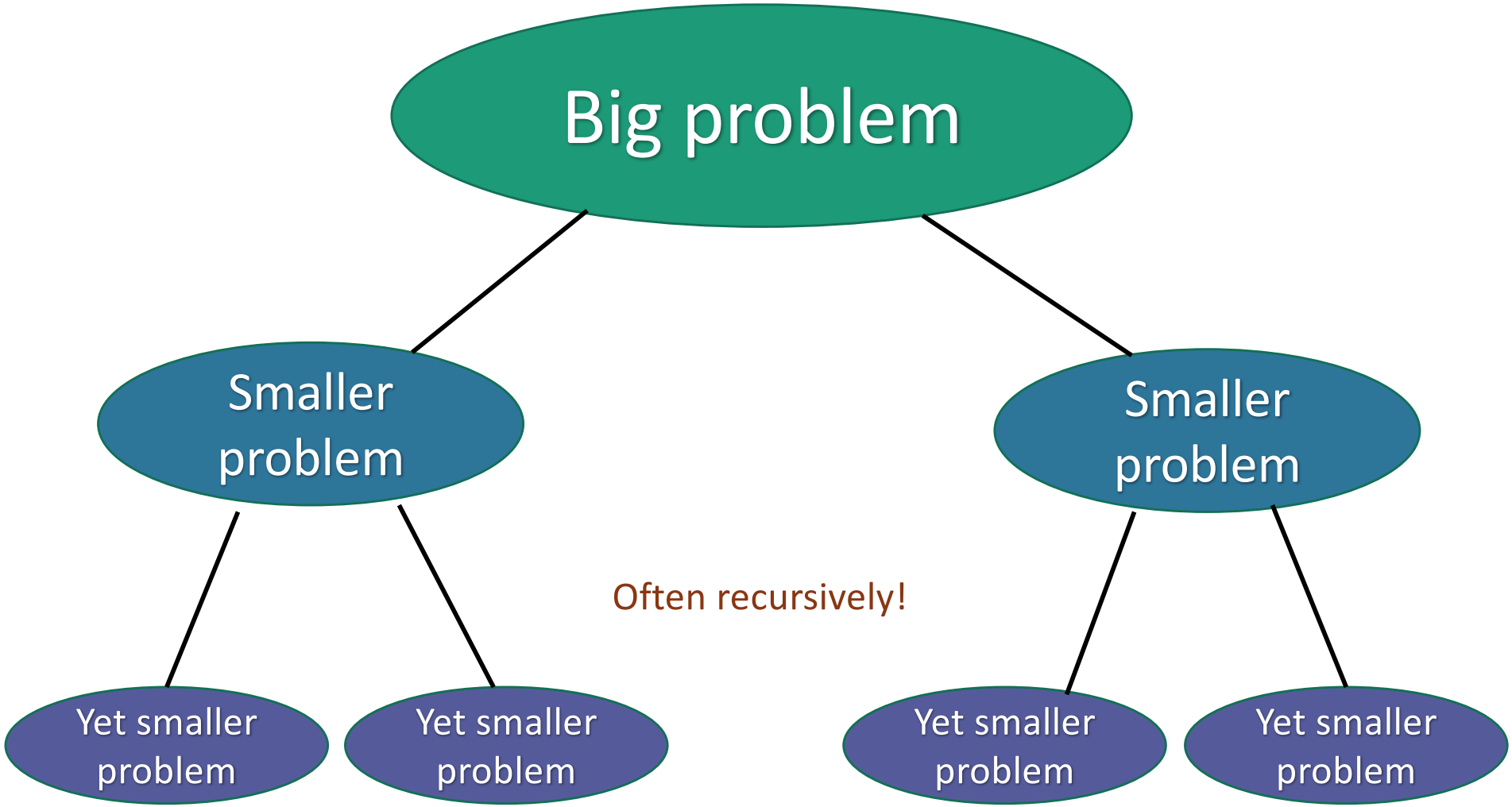
(than n^2?)



Let’s dig in to our algorithmic toolkit…

Divide and conquer

Break problem up into smaller (easier) sub-problems



Divide and conquer for multiplication

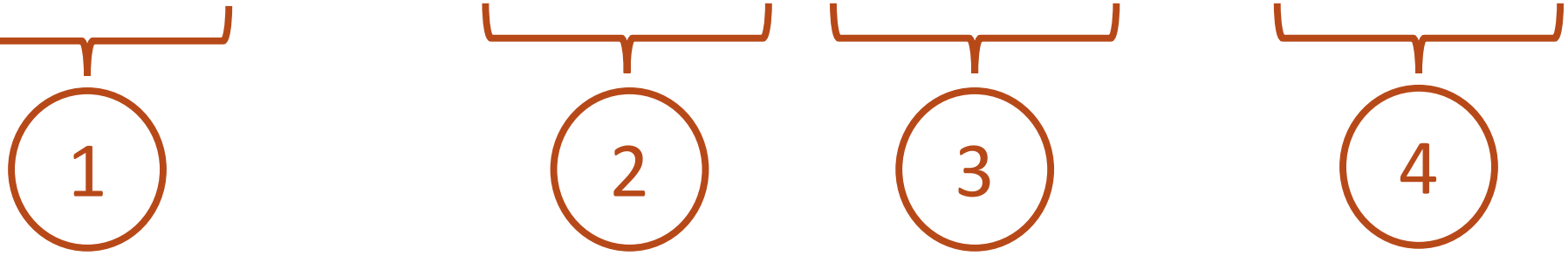
Break up an integer:

$$1234 = 12 \times 100 + 34$$

$$1234 \times 5678$$

$$= (12 \times 100 + 34)(56 \times 100 + 78)$$

$$= (12 \times 56)10000 + (34 \times 56 + 12 \times 78)100 + (34 \times 78)$$



One 4-digit multiply

Four 2-digit multiplies

More generally

Suppose n is even

Break up an n-digit integer:

$$[x_1 x_2 \cdots x_n] = [x_1 x_2 \cdots x_{n/2}] \times 10^{n/2} + [x_{n/2+1} x_{n/2+2} \cdots x_n]$$

$$x \times y = (a \times 10^{n/2} + b)(c \times 10^{n/2} + d)$$

$$= \underbrace{(a \times c)10^n}_{1} + \underbrace{(a \times d}_{2} + \underbrace{c \times b)10^{n/2}}_{3} + \underbrace{(b \times d)}_{4}$$

One n-digit multiply → Four (n/2)-digit multiplies

Divide and conquer algorithm

x,y are n-digit numbers

Multiply(x, y):

- **If** n=1:
  - **Return** xy

Base case: I've memorized my 1-digit multiplication tables…

Say n is even…

- Write $x = a\ 10^{\frac{n}{2}} + b$
- Write $y = c\ 10^{\frac{n}{2}} + d$

a, b, c, d are n/2-digit numbers

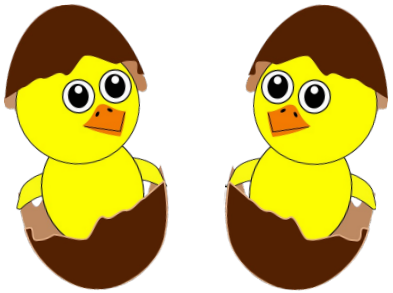
- Recursively compute ac, ad, bc, bd:
  - ac = **Multiply**(a, c), etc…
- Add them up to get xy:
  - xy = ac 10^n + (ad + bc) $10^{n/2}$ + bd

Make this pseudocode more detailed! How should we handle odd n? How should we implement "multiplication by 10^n"?

Siggi the Studious Stork

How long does this take?

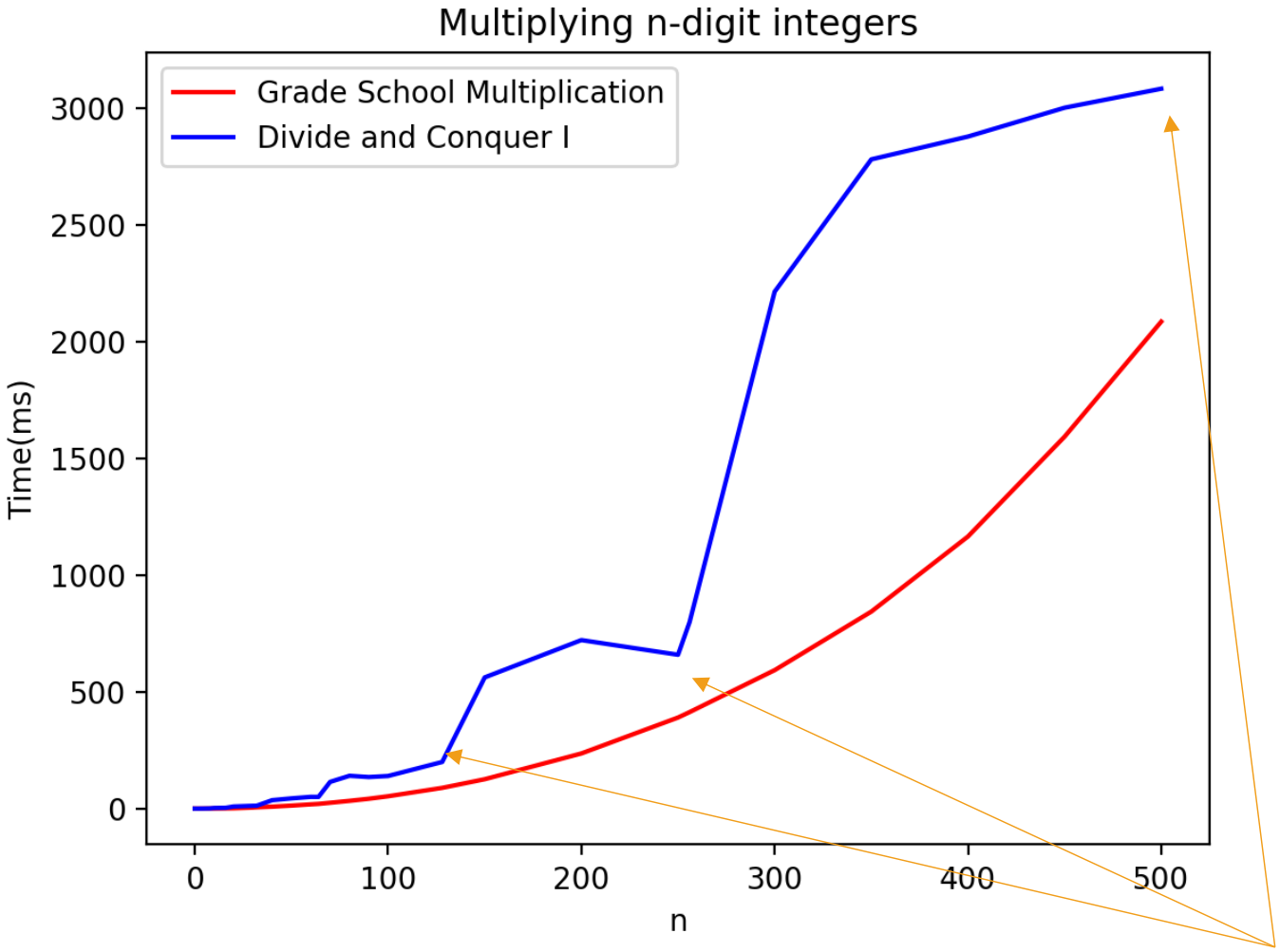
- Better or worse than the grade school algorithm?
  - That is, does the number of operations grow like n^2 ?
  - More or less than that?



Think-Pair-Share:
(2 min: try to think- how fast is our new algorithm?
2 min: what does the person next to you think? why?)

- How do we answer this question?
  1. Try it.
  2. Try to understand it analytically.

1. Try it.



Conjectures about running time?

Doesn't look too good but hard to tell…

Concerns with the conclusiveness of this approach?

Maybe one implementation is slicker than the other?

Maybe if we were to run it to n=10000, things would look different.

Something funny is happening at powers of 2…

2. Try to understand the running time analytically

- Proof by meta-reasoning:

It must be faster than the grade school algorithm, because we are learning it in an algorithms class.

Not sound logic!

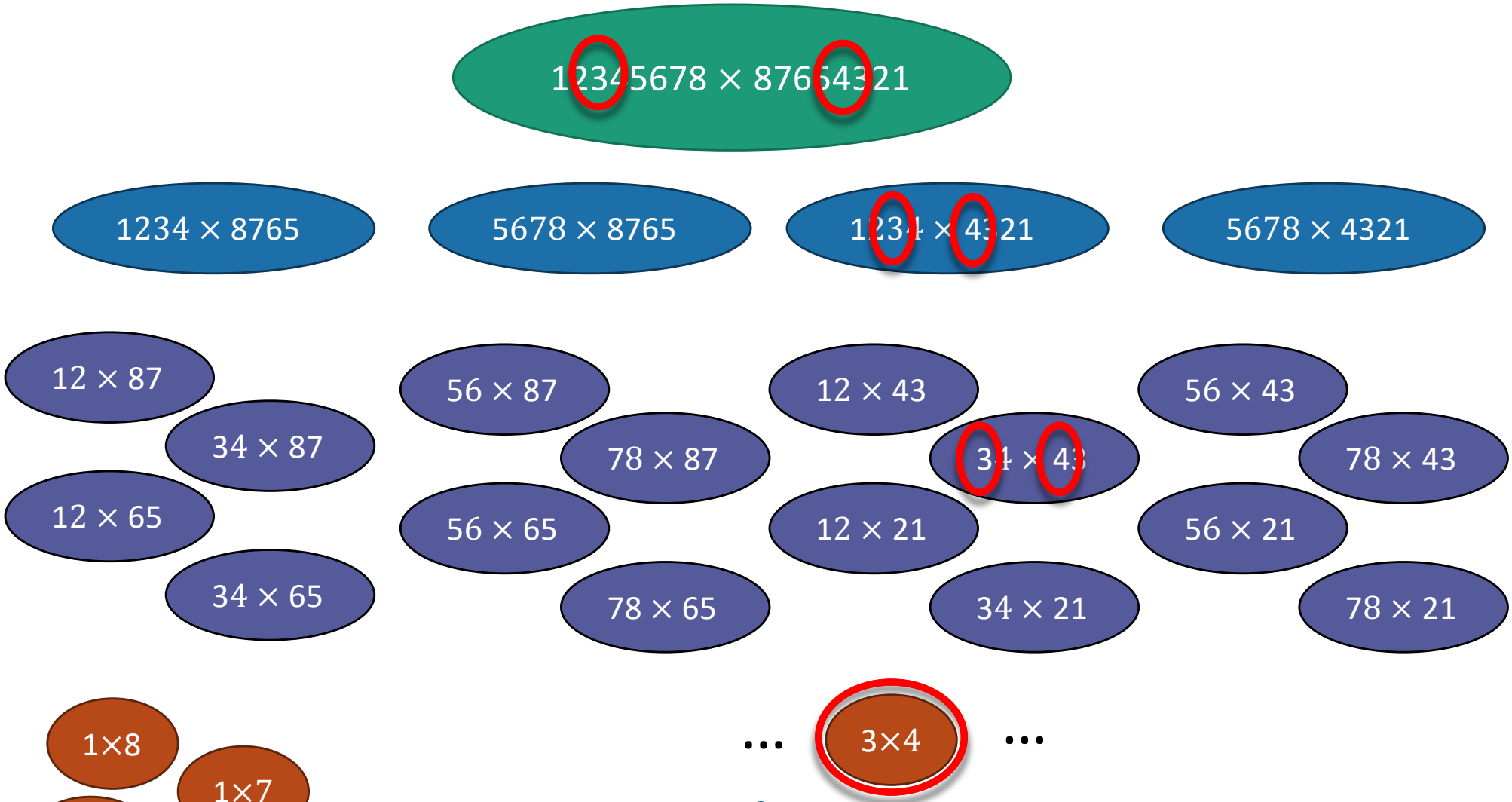
Plucky the Pedantic Penguin

2. Try to understand the running time analytically

- Claim:

The running time of this algorithm is AT LEAST n^2 operations.

How many one-digit multiplies?



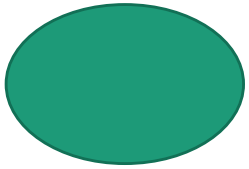
Claim: there are n^2 one-digit problems.

Every pair of digits still gets multiplied together separately.

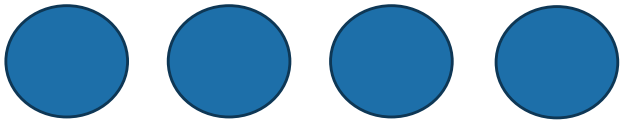
So the running time is still at least n^2.

Another way to see this*

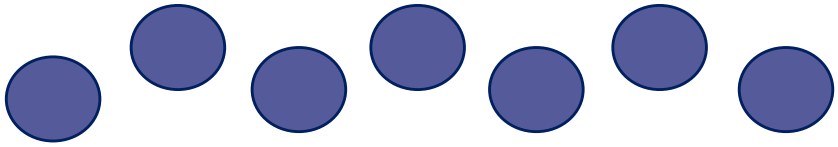
*we will come back to this sort of analysis later and still more rigorously.



1 problem
of size n

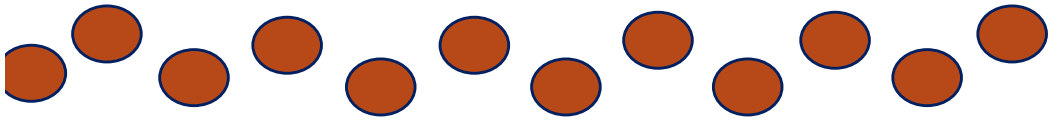


4 problems
of size n/2



4^t problems
of size $n/2^t$

...



$\frac{n^2}{}$ problems
of size 1

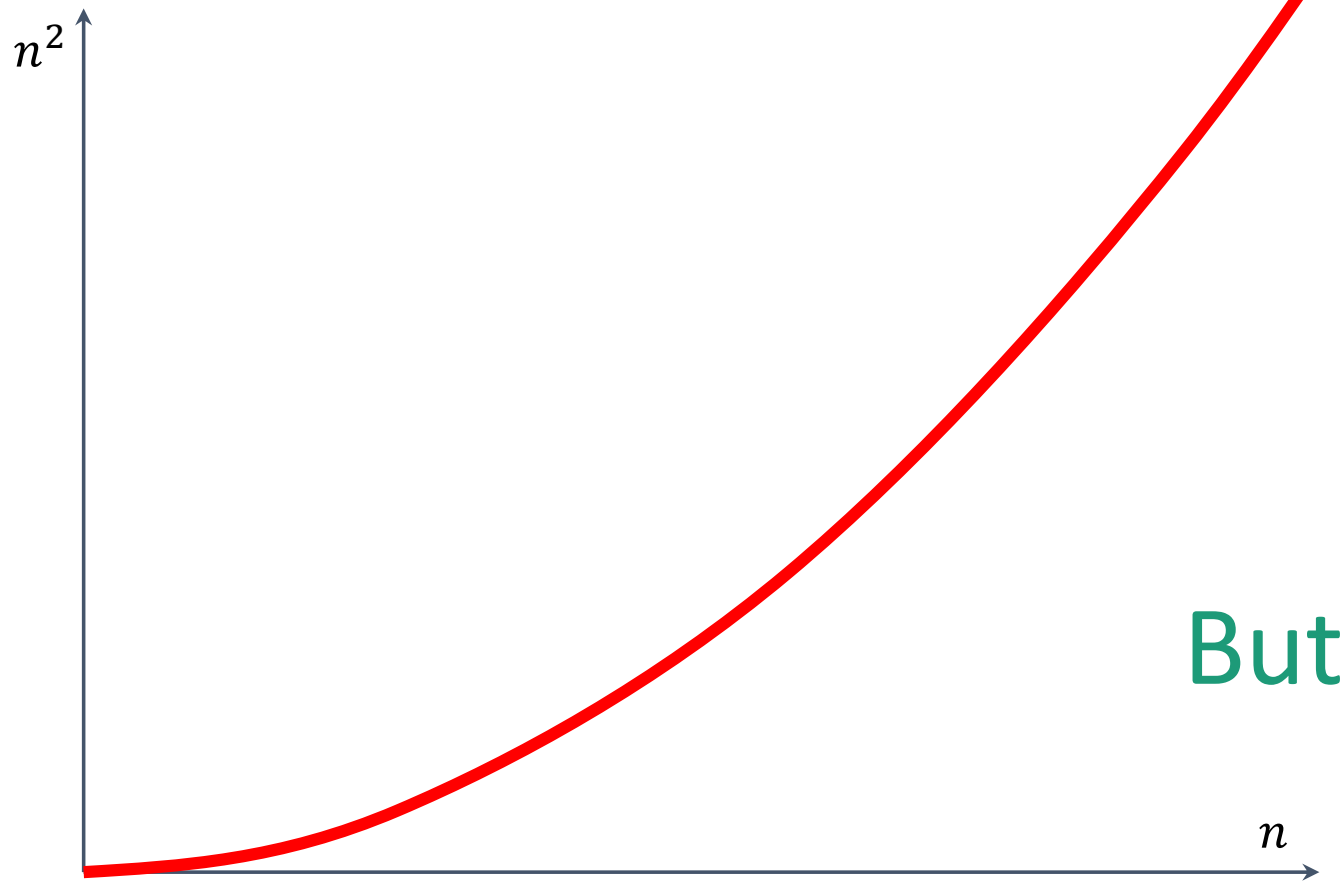
- If you cut n in half $\log_2(n)$ times, you get down to 1.
- So we do this $\log_2(n)$ times and get…

$4^{\log\_2(n)} = n^2$
problems of size 1.

This is just a lower bound – we're just counting the number of size-1 problems!

That’s a bit disappointing

All that work and still (at least) n^2…



But wait!!

Divide and conquer can actually make progress

- Karatsuba figured out how to do this better!

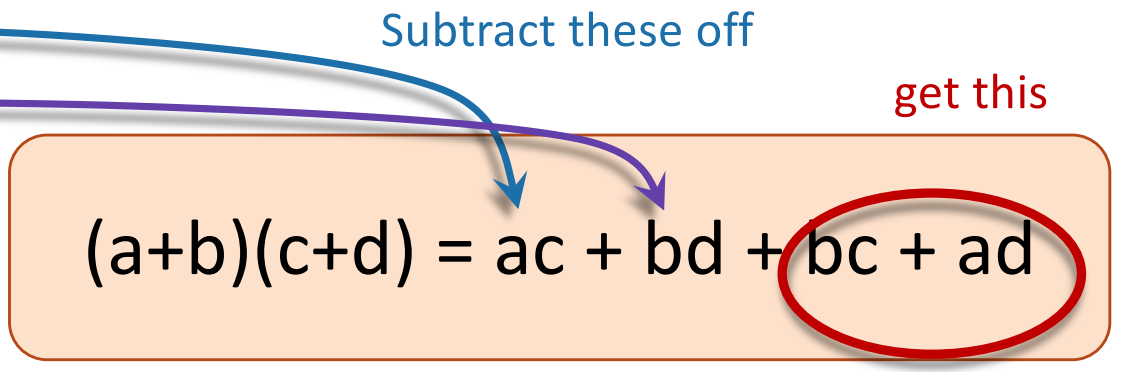
$$xy = (a \cdot 10^{n/2} + b)(c \cdot 10^{n/2} + d)$$
$$= ac \cdot 10^{n} + (ad + bc)10^{n/2} + bd$$

Need these three things

- If only we recurse three times instead of four…

Karatsuba integer multiplication

- Recursively compute these THREE things:
  - ac
  - bd
  - (a+b)(c+d)

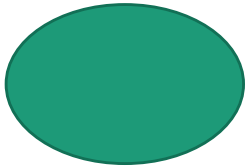


- Assemble the product:

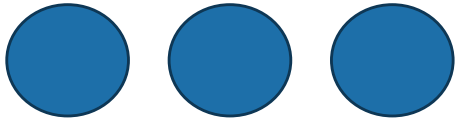
$$xy = (a \cdot 10^{n/2} + b)(c \cdot 10^{n/2} + d)$$
$$= ac \cdot 10^{n} + (ad + bc)10^{n/2} + bd$$

✓ ✓ ✓

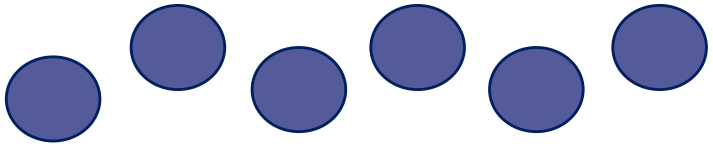
What's the running time?



1 problem
of size n

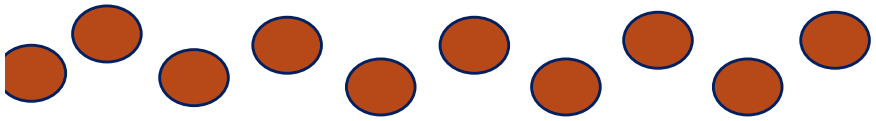


3 problems
of size n/2



3^t problems
of size $n/2^t$

...



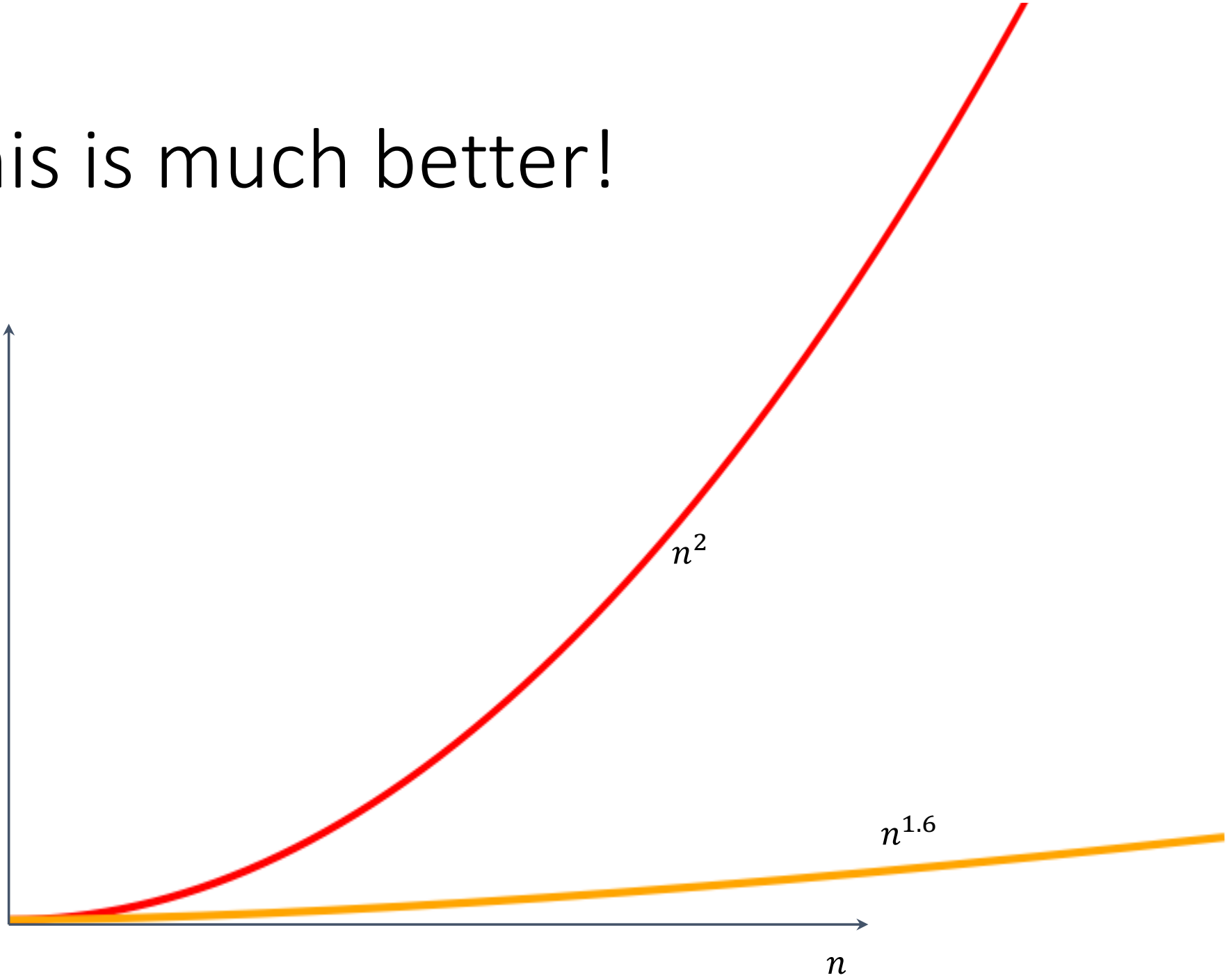
$\frac{n^{1.6}}{}$ problems
of size 1

- If you cut n in half $\log_2(n)$ times, you get down to 1.
- So we do this $\log_2(n)$ times and get…

$3^{\log\_2(n)} = n^{\log\_2(3)} \approx n^{1.6}$
problems of size 1.

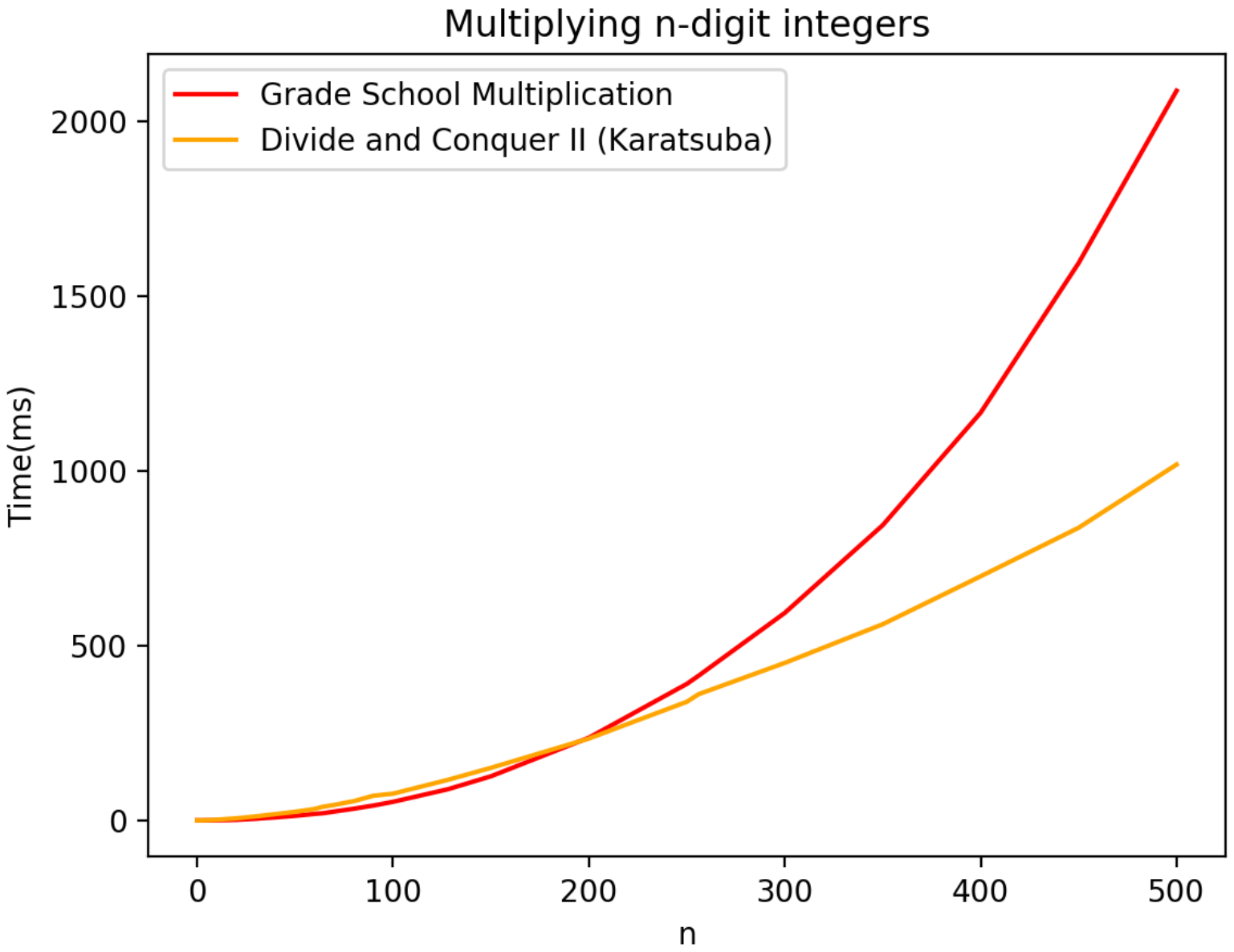
We still aren't accounting for the work at the higher levels! But we'll see later that this turns out to be okay.

This is much better!



We can even see it in real life!

python™



Can we do better?

- **Toom-Cook** (1963): instead of breaking into three n/2-sized problems, break into five n/3-sized problems.
  - This scales like $n^{1.465}$

Try to figure out how to break up an n-sized problem into five n/3-sized problems! **(Hint: start with nine n/3-sized problems).**

Ollie the Over-achieving Ostrich

Given that you can break an n-sized problem into five n/3-sized problems, where does the 1.465 come from?

Siggi the Studious Stork

- **Schönhage–Strassen** (1971):
  - Scales like n log(n) loglog(n)
- **Furer** (2007)
  - Scales like $n \log(n)\ 2^{\log^*(n)}$

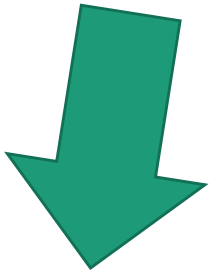
[This is just for fun, you don't need to know these algorithms!]

Course goals

- Think analytically about algorithms
- Flesh out an "algorithmic toolkit"
- Learn to communicate clearly about algorithms

Today's goals

- Karatsuba Integer Multiplication
- Technique: Divide and conquer
- Meta points:
  - How do we measure the speed of an algorithm?



Wrap up

Wrap up

- Algorithms are:
  - Fundamental, useful, and fun!
- In this course, we will develop both algorithmic intuition and algorithmic techniques
  - **It might not be easy but it will be worth it!**
- Karatsuba Integer Multiplication:
  - You can do better than grade school multiplication!
  - Example of divide-and-conquer in action
  - Informal demonstration of asymptotic analysis

Next time

- Sorting!
- Divide and Conquer some more
- Begin Asymptotics and Big-Oh notation